

Assignment 1

Object Oriented Programming

Question #1:

You are designing a class Book for a library system. Each book has a **title**, **author**, **price**, **Available Quantity**, and **Category** of the book. You also need a function that calculates a **discounted price**, and **Order a Book for purchase** based on a given discount percentage. Make sure all data members should not be accessible for outer programming world. 1 constructor set all data member default values, and another should initialize it to fixed values.

Code:

Book.h

```
#include <iostream>
#include <cstring>
#include <iomanip>
using namespace std;
class Book {
private:
    char* m_title;
    char* m_author;
    char* m_category;
    double m_price;
    int m_quantity;
public:
    Book();
```

```
Book(const char*, const char*, const char*, double, int);  
~Book();  
  
void setTitle(const char*);  
void setAuthor(const char*);  
void setCategory(const char*);  
void setPrice(double);  
void setQuantity(int);  
  
char* getTitle();  
char* getAuthor();  
char* getCategory();  
double getPrice();  
int getQuantity();  
  
void orderBook(int);  
double calculateDiscount(int);  
  
void printBook();  
};
```

Book.cpp

```
#include "Book.h"
```

```
Book::Book() {
```

```
    m_title = nullptr;
```

```
    m_author = nullptr;
```

```
    m_category = nullptr;
```

```
    m_price = 0.0;
```

```
    m_quantity = 0;
```

```
}
```

```
Book::Book(const char* title, const char* author, const char* category, double  
price, int quantity) {
```

```
    m_title = new char[strlen(title) + 1];
```

```
    strcpy(m_title, title);
```

```
    m_author = new char[strlen(author) + 1];
```

```
    strcpy(m_author, author);
```

```
    m_category = new char[strlen(category) + 1];
```

```
    strcpy(m_category, category);
```

```
    m_price = price;
```

```
    m_quantity = quantity;
```

```
}
```

```
Book::~~Book() {
```

```
    delete[] m_title;
```

```
    delete[] m_author;
```

```
    delete[] m_category;
```

```
}
```

```
void Book::setTitle(const char* title) {
```

```
    delete[] m_title;
```

```
    m_title = new char[strlen(title) + 1];
```

```
    strcpy(m_title, title);
```

```
}
```

```
void Book::setAuthor(const char* author) {
```

```
    delete[] m_author;
```

```
    m_author = new char[strlen(author) + 1];
```

```
    strcpy(m_author, author);
```

```
}
```

```
void Book::setCategory(const char* category) {
```

```
    delete[] m_category;
```

```
    m_category = new char[strlen(category) + 1];
```

```
    strcpy(m_category, category);
```

```
}

void Book::setPrice(double price) {
    m_price = price;
}

void Book::setQuantity(int quantity) {
    m_quantity = quantity;
}

char* Book::getTitle() {
    if (m_title == nullptr) return nullptr;
    char* temp = new char[strlen(m_title) + 1];
    strcpy(temp, m_title);
    return temp;
}

char* Book::getAuthor() {
    if (m_author == nullptr) return nullptr;
    char* temp = new char[strlen(m_author) + 1];
    strcpy(temp, m_author);
    return temp;
}
```

```
char* Book::getCategory() {  
    if (m_category == nullptr) return nullptr;  
    char* temp = new char[strlen(m_category) + 1];  
    strcpy(temp, m_category);  
    return temp;  
}  
  
double Book::getPrice() {  
    return m_price;  
}  
  
int Book::getQuantity() {  
    return m_quantity;  
}  
  
double Book::calculateDiscount(int disc) {  
    return m_price - (m_price * disc / 100);  
}  
  
void Book::orderBook(int quantity) {  
    if (quantity > m_quantity) {  
        cout << "Not enough stock available! Only " << m_quantity << " books left."  
        << endl;  
        return;  
    }  
}
```

```

cout << "Do you want to redeem discount? (y/n): " << endl;
char choice;
cin >> choice;

double total = 0.0;
if (choice == 'y' || choice == 'Y') {
    int disc;
    cout << "Enter discount percentage: ";
    cin >> disc;
    total = calculateDiscount(disc) * quantity;
} else {
    total = m_price * quantity;
}
m_quantity -= quantity;
cout << "Order placed successfully!" << endl;
cout << "Total amount: " << total << endl;
cout << "Remaining stock: " << m_quantity << endl;
}

void Book::printBook() {
    //This function can also get values without getters
    cout << "Title: " << getTitle() << endl;
    cout << "Author: " << getAuthor() << endl;
}

```

```
cout << "Category: " << getCategory() << endl;  
cout << "Price: " << fixed << setprecision(3) << getPrice() << endl;  
cout << "Quantity: " << getQuantity() << endl;  
}
```


Void Book

```
#include "Book.h"
```

```
void book() {
```

```
    system("cls");
```

```
    Book b_1("Udaas Naslen", "Abdullah Hussain", "Fiction", 500, 20);
```

```
    Book b_2;
```

```
    cout << "Enter book title" << endl;
```

```
    char* title = new char[20];
```

```
    cin.ignore();
```

```
    cin.getline(title, 20);
```

```
    b_2.setTitle(title);
```

```
    cout << "Enter book author" << endl;
```

```
    char* author = new char[20];
```

```
    cin.getline(author, 20);
```

```
    b_2.setAuthor(author);
```

```
    cout << "Enter book category" << endl;
```

```
    char* category = new char[20];
```

```
    cin.getline(category, 20);
```

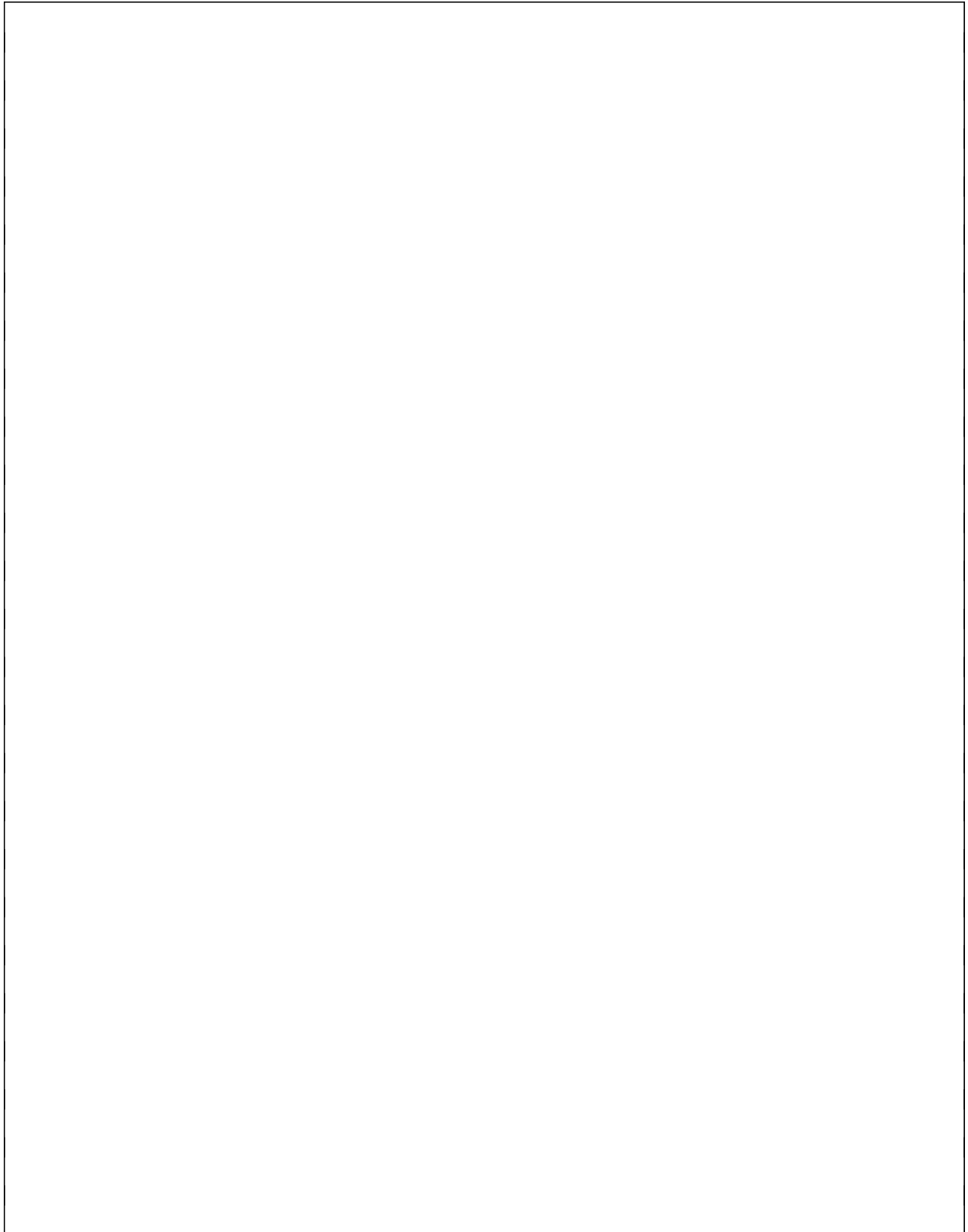
```
    b_2.setCategory(category);
```

```
cout << "Enter book price" << endl;
double price;
cin >> price;
b_2.setPrice(price);

cout << "Enter book quantity" << endl;
int quantity;
cin >> quantity;
b_2.setQuantity(quantity);
system("cls");
cout << "Book 1 with parameterized constructor: " << endl;
b_1.printBook();
cout << endl << "Book 2 with updated values: " << endl;
b_2.printBook();

cout << "How many books do you want to order? " << endl;
int n;
cin >> n;
b_2.orderBook(n);

system("pause");
}
```



Screenshot:

This screenshot shows the Visual Studio Code editor with a C++ file named `task_1.cpp` open. The Explorer sidebar on the left shows the project structure, including `task_1`, `task_2`, and `assignment-1.docx`. The main editor area displays the following code:

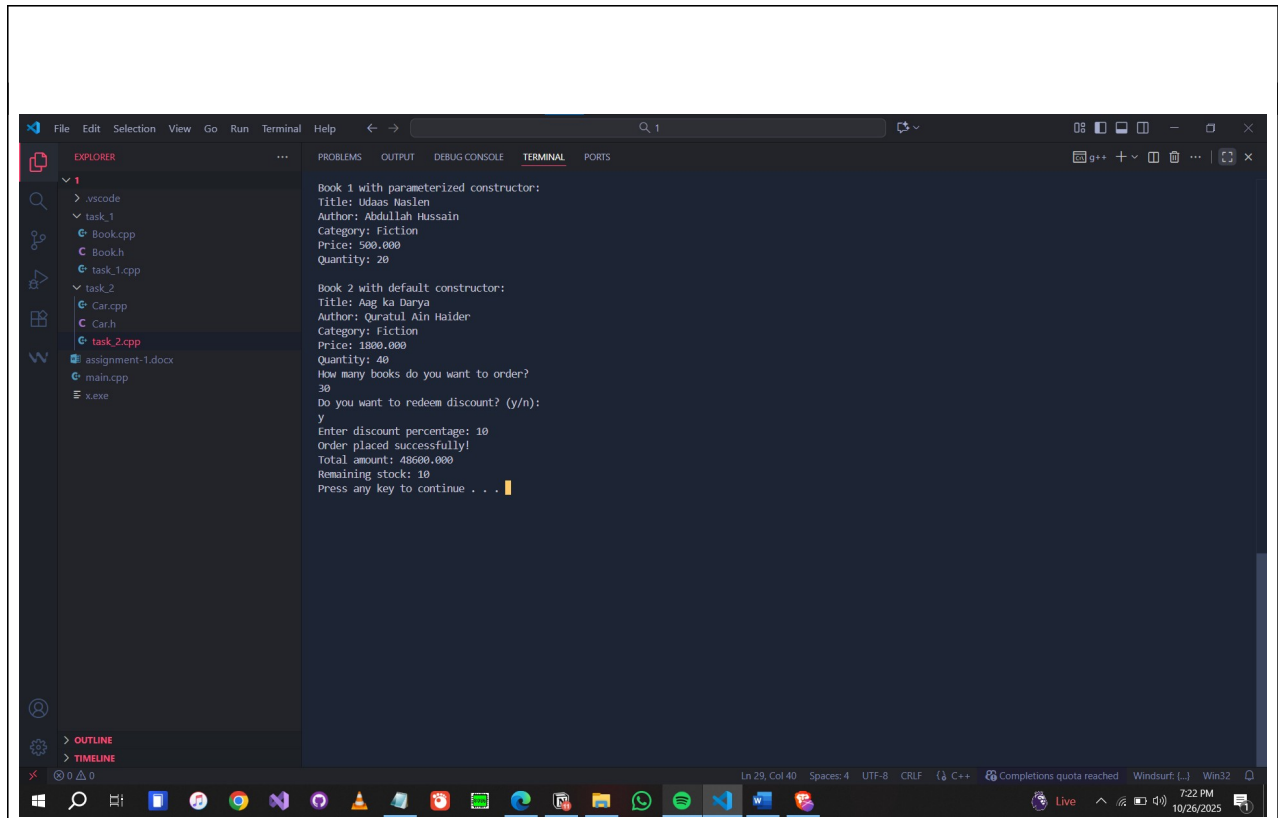
```
*** Menu ***
1. Book
2. Car
0. Exit
Enter your choice (1 - 2): 1
```

The status bar at the bottom indicates the file is at line 46, column 2, with 4 spaces, UTF-8 encoding, and CRLF line endings. The Windows taskbar at the bottom shows the time as 7:07 PM on 10/26/2025.

This screenshot shows the same Visual Studio Code editor with the `task_1.cpp` file. The program has executed, and the user has entered the following input:

```
Enter book title
Aag ka Darya
Enter book author
Quratul Ain Haider
Enter book category
Fiction
Enter book price
1800
Enter book quantity
4
```

The status bar at the bottom shows the time as 7:09 PM on 10/26/2025.



Question #2:

Write a class Car that has:

- Data members: brand, model, and price.
- A **default constructor** that sets default values ("Unknown", "N/A", 0).
- Constructor Overloads take all values
- A **function** display() that shows all details.

Task:

1. Create two objects: one using the default constructor and another using a **parameterized constructor**.
2. Display both cars' details.
3. Use getter and setter function to set and get the car attributes values

Code:

Car.h

```
#include <iostream>
#include <cstring>
#include <iomanip>
using namespace std;
class Car {
private:
    char* m_brand;
    char* m_model;
    int m_year;
    double m_price;
public:
```

```
Car();
```

```
Car(const char*, const char*, int, double);
```

```
~Car();
```

```
void setValue(char*, char*, int, double);
```

```
char* getBrand();
```

```
char* getModel();
```

```
int getYear();
```

```
double getPrice();
```

```
void display();
```

```
};
```

Car.cpp

```
#include "Car.h"
```

```
Car::Car() {
```

```
    m_brand = new char[strlen("Unknown") + 1];
```

```
    strcpy(m_brand, "Unknown");
```

```
    m_model = new char[strlen("N/A") + 1];
```

```
    strcpy(m_model, "N/A");
```

```
    m_year = 0;
```

```
    m_price = 0.0;
```

```
}
```

```
Car::Car(const char* brand, const char* model, int year, double price) {
```

```
    m_brand = new char[strlen(brand) + 1];
```

```
    strcpy(m_brand, brand);
```

```
    m_model = new char[strlen(model) + 1];
```

```
    strcpy(m_model, model);
```

```
    m_year = year;
```

```
    m_price = price;
```

```
}
```



```
Car::~~Car() {
    delete[] m_brand;
    delete[] m_model;
}

void Car::setValue(char* brand, char* model, int year, double price) {
    delete[] m_brand;
    m_brand = new char[strlen(brand) + 1];
    strcpy(m_brand, brand);

    delete[] m_model;
    m_model = new char[strlen(model) + 1];
    strcpy(m_model, model);

    m_year = year;
    m_price = price;
}

char* Car::getBrand() {
    if (m_brand == nullptr) return nullptr;
    char* temp = new char[strlen(m_brand) + 1];
    strcpy(temp, m_brand);
    return temp;
}
```

```
}
```

```
char* Car::getModel() {
```

```
    if (m_model == nullptr) return nullptr;
```

```
    char* temp = new char[strlen(m_model) + 1];
```

```
    strcpy(temp, m_model);
```

```
    return temp;
```

```
}
```

```
int Car::getYear() {
```

```
    return m_year;
```

```
}
```

```
double Car::getPrice() {
```

```
    return m_price;
```

```
}
```

```
void Car::display() {
```

```
    //This function can also get values without getters
```

```
    cout << "Brand: " << getBrand() << endl;
```

```
    cout << "Model: " << getModel() << endl;
```

```
    cout << "Year: " << getYear() << endl;
```

```
    cout << "Price: Rs. " << getPrice() << endl;
```

```
}
```

Void Car

```
#include "Car.h"
```

```
void car() {
```

```
    system("cls");
```

```
    Car c_1("Mercedes", "Benz", 2022, 1000000);
```

```
    Car c_2;
```

```
    cout << "Car 1 with parameterized constructor: " << endl;
```

```
    c_1.display();
```

```
    cout << endl << "Car 2 with default constructor: " << endl;
```

```
    c_2.display();
```

```
    system("pause");
```

```
    char* brand = new char[20];
```

```
    char* model = new char[20];
```

```
    int year = 0;
```

```
    double price = 0;
```

```
    cout << endl << "Enter car brand: " << endl;
```

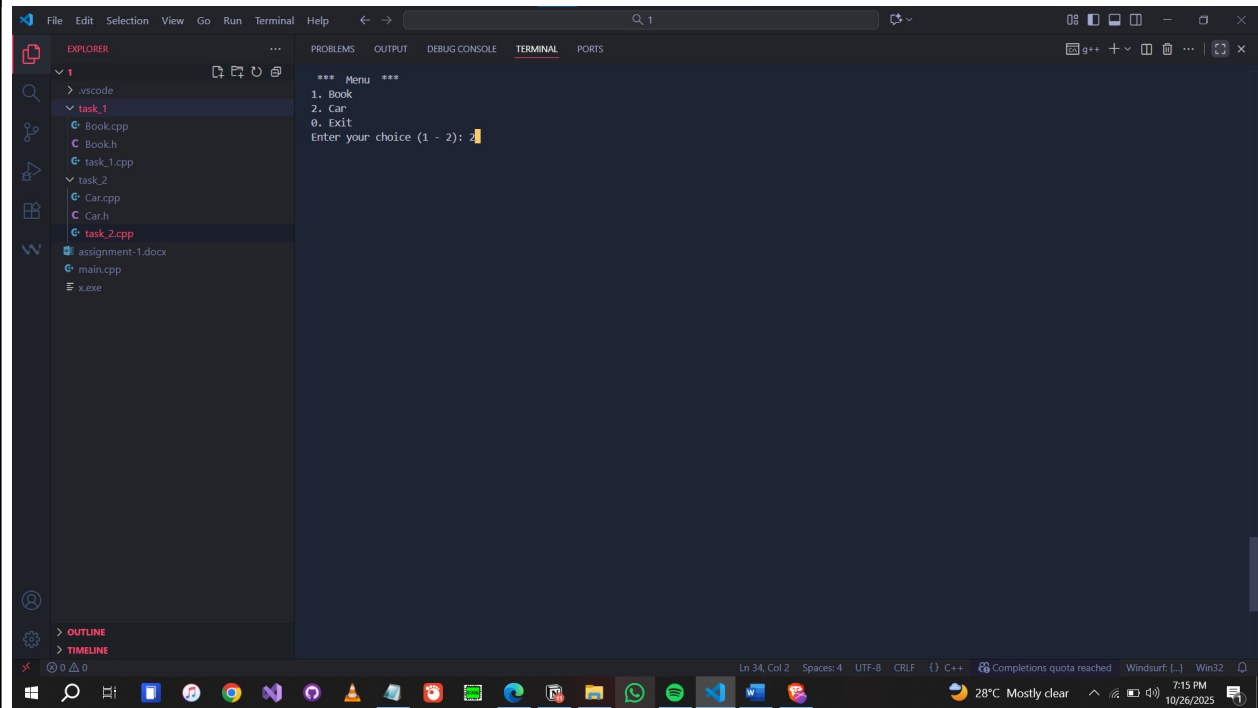
```
    cin.ignore();
```

```
    cin.getline(brand, 20);
```

```
    cout << "Enter car model: " << endl;
```

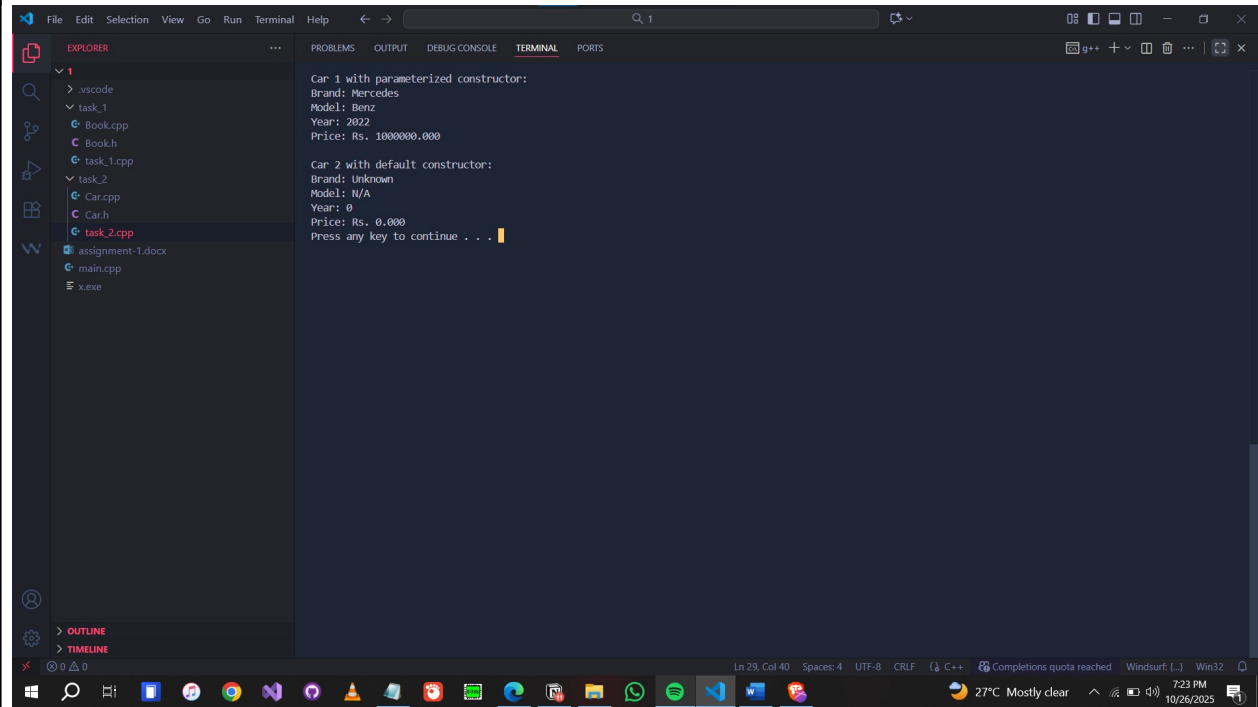
```
cin.getline(model, 20);  
cout << "Enter car year: " << endl;  
cin >> year;  
cout << "Enter car price: " << endl;  
cin >> price;  
  
c_2.setValue(brand, model, year, price);  
  
cout << endl << "Car 2 with updated values: " << endl;  
c_2.display();  
  
system("pause");  
}
```

Screenshot:



This screenshot shows the Visual Studio Code editor with a C++ project. The Explorer sidebar on the left shows the file structure: .vscode, task_1, task_2, and task_2.cpp (selected). The main editor displays the code for task_2.cpp, which is a menu program. The terminal at the bottom shows the program's output.

```
*** Menu ***
1. Book
2. Car
0. Exit
Enter your choice (1 - 2): 2
```



This screenshot shows the same Visual Studio Code editor with the same project. The terminal output now shows the results of creating two car objects. The first car is created with a parameterized constructor, and the second car is created with a default constructor. The program then prompts the user to press any key to continue.

```
Car 1 with parameterized constructor:
Brand: Mercedes
Model: Benz
Year: 2022
Price: Rs. 1000000.000

Car 2 with default constructor:
Brand: Unknown
Model: N/A
Year: 0
Price: Rs. 0.000
Press any key to continue . . .
```

